



UDEM

UNIVERSIDAD DE MONTERREY

Práctica 3

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 5 de Junio del 2026

Introducción

El display de siete segmentos es uno de los dispositivos de visualización numérica más utilizados en sistemas embebidos. Está compuesto por siete LEDs individuales organizados en forma de dígito, etiquetados convencionalmente como segmentos a, b, c, d, e, f y g, cuya activación combinada permite representar los dígitos del 0 al 9 y varios caracteres hexadecimales. En esta práctica se empleó el PIC16F887 operando con un cristal de cuarzo de 16 MHz para controlar dicho display a través del Puerto D, con el objetivo de familiarizarse con el manejo de salidas digitales, la codificación de patrones de bits y la temporización por software.

Se plantearon dos actividades progresivas. La primera consistió en implementar un contador del 0 al 9, lo que exigió definir los bytes de patrón para cada dígito decimal y enviarlos secuencialmente al puerto de salida con un retardo entre cada valor. La segunda actividad extendió la secuencia para incluir, después del 9, los seis caracteres hexadecimales A, b, C, d, E y F, completando así el recorrido completo de los 16 valores del sistema hexadecimal en un único arreglo de datos.

Ambas actividades se implementaron en lenguaje C con MPLAB X IDE y el compilador XC8, verificándose primero en simulación con Proteus antes de armarse en el protoboard físico. Un aspecto clave del desarrollo fue la correcta definición de los bits de configuración del microcontrolador, ya que un error en una de esas declaraciones puede impedir que el programa ejecute correctamente incluso cuando la lógica del código es válida, algo que el equipo experimentó directamente durante la sesión.

Marco Teórico

Display de Siete Segmentos

Un display de siete segmentos es un dispositivo optoelectrónico compuesto por siete diodos emisores de luz dispuestos en una configuración estándar que permite representar visualmente caracteres numéricos y alfanuméricos. Cada segmento está

identificado con una letra de la a a la g, y su encendido selectivo genera los distintos dígitos y caracteres. Existen dos configuraciones eléctricas: ánodo común, donde todos los ánodos comparten una misma línea de alimentación y los segmentos se activan con un nivel lógico bajo, y cátodo común, donde los cátodos comparten tierra y los segmentos se activan con un nivel lógico alto. En esta práctica se utilizó la configuración de cátodo común, por lo que un "1" en el bit correspondiente enciende el segmento asociado [1].

La correspondencia entre segmentos y bits del byte de control se establece asignando cada bit a un segmento específico. En la implementación de esta práctica, los siete bits superiores del Puerto D se mapearon a los segmentos del display, con el bit más significativo correspondiendo al segmento a y los demás siguiendo el orden convencional. El byte 0xFC, por ejemplo, activa los segmentos a, b, c, d, e y f dejando apagado el segmento g, lo que produce el dígito 0. Esta codificación directa en un arreglo permite al microcontrolador recorrer los patrones con un simple índice, sin necesidad de lógica adicional de conversión [2].

Puerto D del PIC16F887 y Bits de Configuración

El Puerto D del PIC16F887 es un puerto bidireccional de ocho bits controlado por el registro TRISD. Al escribir 0x00 en TRISD, todos los pines del puerto quedan configurados como salidas digitales, permitiendo que los valores escritos en PORTD se reflejen directamente en los pines físicos del microcontrolador. Esta configuración es la adecuada para manejar un display de siete segmentos, ya que cada bit del registro PORTD controla directamente uno de los segmentos del display a través de una resistencia limitadora de corriente [2].

Los bits de configuración del PIC16F887 se declaran mediante directivas `#pragma config` al inicio del código y determinan el comportamiento fundamental del microcontrolador antes de que el programa comience a ejecutarse. Entre los más relevantes para esta práctica se encuentran FOSC, que selecciona el modo del oscilador, configurado en XT para operar con el cristal de cuarzo de 16 MHz; WDTE, que habilita o deshabilita el Watchdog Timer, desactivado en este caso para evitar reinicios no deseados durante las pruebas; y LVP, que controla la programación de bajo voltaje, también desactivado. Un

error en cualquiera de estas declaraciones puede provocar que el microcontrolador no ejecute el programa correctamente aunque el código de usuario sea válido [2].

Actividad 1 — Contador 0 a 9

Para implementar el contador decimal se define un arreglo de diez bytes donde cada elemento contiene el patrón de bits correspondiente a un dígito del 0 al 9. El bucle principal recorre el arreglo secuencialmente, escribe cada byte en PORTD y espera 200 ms antes de avanzar al siguiente valor. Este retardo, generado por la función `__delay_ms()` del compilador XC8 calibrada con `_XTAL_FREQ = 16000000`, garantiza que cada dígito sea legible visualmente antes de la transición al siguiente. Al alcanzar el índice 9, el bucle reinicia desde cero produciendo un conteo cíclico continuo [1].

Actividad 2 — Contador 0 a F (Hexadecimal)

La segunda actividad extiende el arreglo de diez a dieciséis elementos añadiendo los seis caracteres hexadecimales A, b, C, d, E y F después del dígito 9. La elección de mayúsculas y minúsculas en estos caracteres no es arbitraria: se utiliza b minúscula en lugar de B mayúscula y d minúscula en lugar de D mayúscula porque la representación de B mayúscula es idéntica al 8, y la de D mayúscula es indistinguible del 0 en un display de siete segmentos. Esta distinción es una práctica estándar en diseño de displays y permite que el recorrido hexadecimal completo sea legible sin ambigüedad. El comportamiento del bucle es idéntico al de la primera actividad, con la única diferencia de que el límite del índice se extiende de 10 a 16 [1].

Simulación

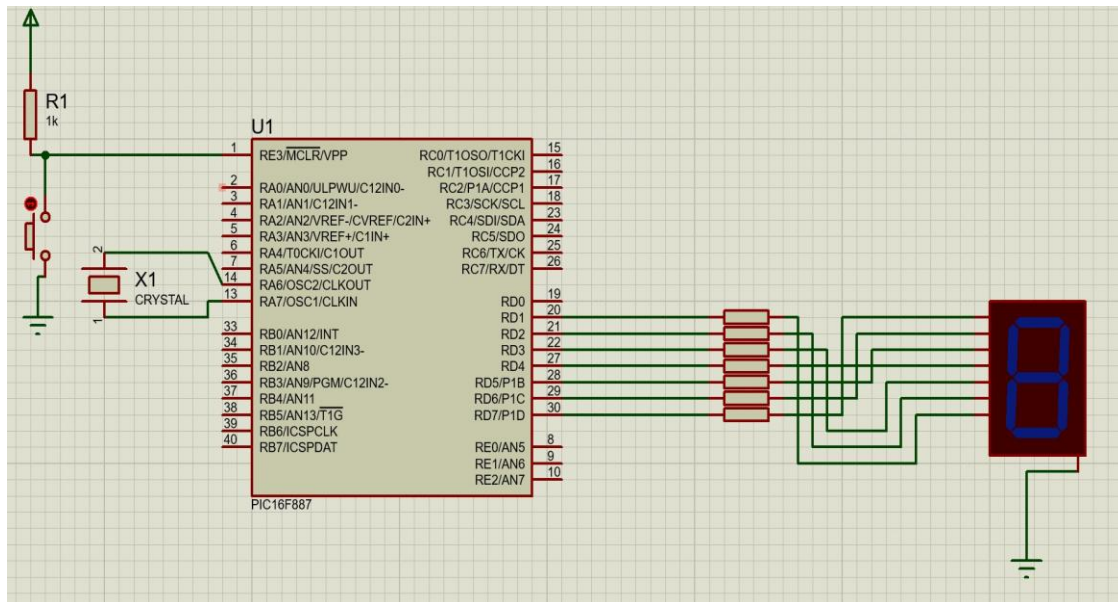
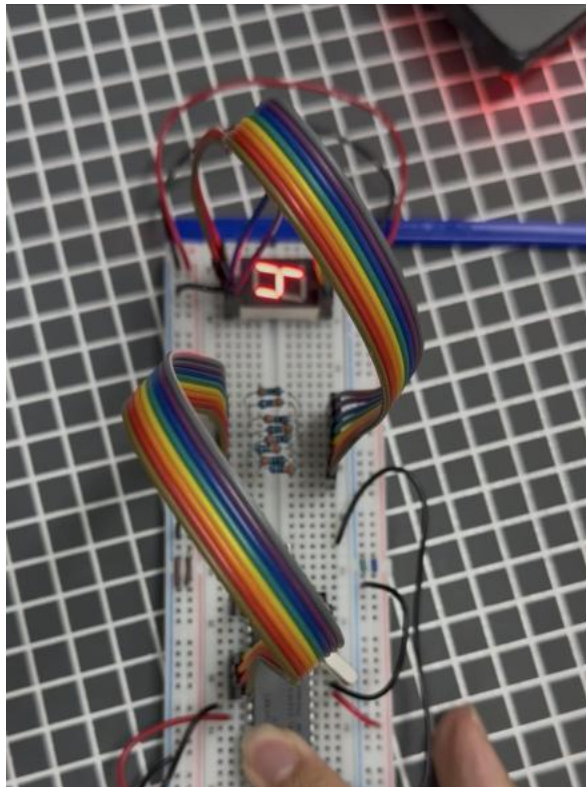


Foto del Circuito en Físico



Resultados

En la Actividad 1, el contador decimal funcionó correctamente desde la primera prueba en el circuito físico. Los dígitos del 0 al 9 se mostraron de forma secuencial con una cadencia de 200 ms por dígito, produciendo una transición fluida y legible. Cada patrón de bits del arreglo encendió los segmentos correctos del display, confirmando que el mapeo entre los bits del Puerto D y los segmentos físicos del componente era el esperado.

En la Actividad 2, la secuencia extendida del 0 al F también operó sin contratiempos una vez corregido el único problema encontrado durante el desarrollo: un error en la declaración de los bits de configuración que impedía la ejecución correcta del programa. Al identificar y corregir dicha declaración, el microcontrolador retomó su funcionamiento normal y la secuencia hexadecimal completa se desplegó tal como estaba diseñada. Los caracteres A, b, C, d, E y F resultaron claramente distinguibles en el display gracias a la elección intencional de mayúsculas y minúsculas en los patrones.

En términos generales, ambas actividades concluyeron al 100%. La práctica puso de manifiesto que la depuración en sistemas embebidos muchas veces no está en la lógica del programa sino en la configuración del hardware por software, y que revisar los bits de configuración es un paso previo indispensable antes de proceder a depurar el código de usuario.

Conclusiones Individuales

Ricardo Garza:

Lo más importante que me llevo de esta práctica es que revisar el código con cuidado antes de cualquier prueba física ahorra una cantidad considerable de tiempo. Un error en la declaración de los bits de configuración me tuvo depurando el circuito físico durante un buen rato antes de darme cuenta de que el problema no era el cableado ni los valores del arreglo, sino una línea de configuración incorrecta al inicio del archivo. A partir de ahora voy a tener el hábito de verificar primero los pragma config antes de trasladar cualquier programa al hardware real, porque ese tipo de errores no generan advertencias del compilador y son difíciles de detectar si no se sabe dónde buscar.

Eugenio Romo:

Esta práctica me permitió entender mejor cómo se traduce un valor numérico a una representación visual a través de un arreglo de patrones de bits. Lo que me pareció más interesante fue la decisión de usar b y d en minúscula para los caracteres hexadecimales, algo que en principio parece un detalle menor pero que en realidad es necesario para que el display sea legible sin ambigüedad. Eso me hizo pensar en que en el diseño de sistemas embebidos hay muchas decisiones que no son de software ni de hardware puro, sino de cómo se presenta la información al usuario final.

Referencias

[1] Merino, L. A. (2007). *Microcontroladores PIC: Diseño práctico de aplicaciones* (2.^a ed.). McGraw-Hill.

[2] Microchip Technology. (2009). *PIC16F887 Data Sheet*.
<https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>